FIFO-RSHD

Tofan Gabriela-Felicia Enache Andrei Văduva Andreea-Elena

Universitatea din Bucuresti
Departamentul de Informatica
Instrumente si tehnici de baza in informatica
2025-2026

Outline

- Descrierea problemei
- Specificația soluției
- Designul sistemului
- Detalii de implementare
- Experimente și rezultate
- Concluzii

Descrierea problemei

1. Contextul problemei

În universul sistemelor de operare Linux, procesele rulează adesea în izolare. Pentru a colabora, acestea au nevoie de mecanisme de comunicare între procese (*Inter-Process Communication*). Unul dintre cele mai fundamentale și eficiente mecanisme este **FIFO** (*First-In, First-Out*), cunoscut și sub numele de **named pipe**. Spre deosebire de pipe-urile obișnuite, fișierele FIFO au un nume în sistemul de fișiere, permițând proceselor care nu au o legătură de rudenie (*părinte-copil*) să schimbe date într-un flux unidirecțional.

2. Definirea Problemei

Problema centrală a acestui proiect este proiectarea și implementarea unui sistem **Remote Shell Daemon** (*RSHD*). Scopul este de a crea o arhitectură client-server capabilă să gestioneze următoarele funcții:

- **Interacțiune multi-client**: Sistemul trebuie să permită mai multor clienți să trimită simultan cereri către serverul central pentru a executa comenzi shell (ex: ls, data, wc, etc.).
- **Execuție la distanță (simulare)**: Serverul trebuie să preia aceste comenzi, să le execute într-un mediu controlat și să returneze rezultatul exact clientului care a inițiat cererea.
- **Format strict al mesajului**: Comunicarea trebuie să respecte un protocol predefinit pentru a asigura integritatea datelor, folosind structura:

```
BEGIN-REQ [client-pid: command] END-REQ
```

Descrierea problemei

3. Miza și Obiectivele Educaționale

- Învățarea modului în care datele pot circula între **multiple scripturi Bash** care rulează în paralel, fără a produce blocaje (*deadlocks*) sau pierderi de informație.

- Implementarea unei strategii de tip **Round-Robin** pentru a împărții munca între mai multe servere *„slave”*, asigurând astfel că niciun proces nu este suprasolicitat în timp ce altele sunt inactive.

- Înțelegerea necesității de a crea și, mai ales, de a șterge fișierele temporare de comunicație (*FIFO-urile personalizate pentru răspuns*) pentru a menține integritatea sistemului de fișiere.

Specificația Soluției

Scopul principal al acestui proiect este proiectarea și implementarea unui sistem distribuit, bazat pe o arhitectură ierarhică formată din trei entități distincte: **Master**, **Slave** și **Client**. Proiectul urmărește construirea unui sistem prin care mai mulți utilizatori (*Clienți*) pot trimite comenzi spre execuție către **un server central** (*Master și Sclavi*), utilizând fișiere speciale de tip **FIFO** pentru o comunicare rapidă și organizată între procese.

1. Client

- **Trimite cererea** într-un format fix: BEGIN-REQ [PID: comandă] END-REQ.
- Își **creează propriul canal de primire** a răspunsului (*un FIFO care are în nume PID-ul lui, de ex: /tmp/server-reply-1234*).
- După ce vede rezultatul pe ecran, **șterge canalul creat** pentru a nu lăsa gunoi în sistem.

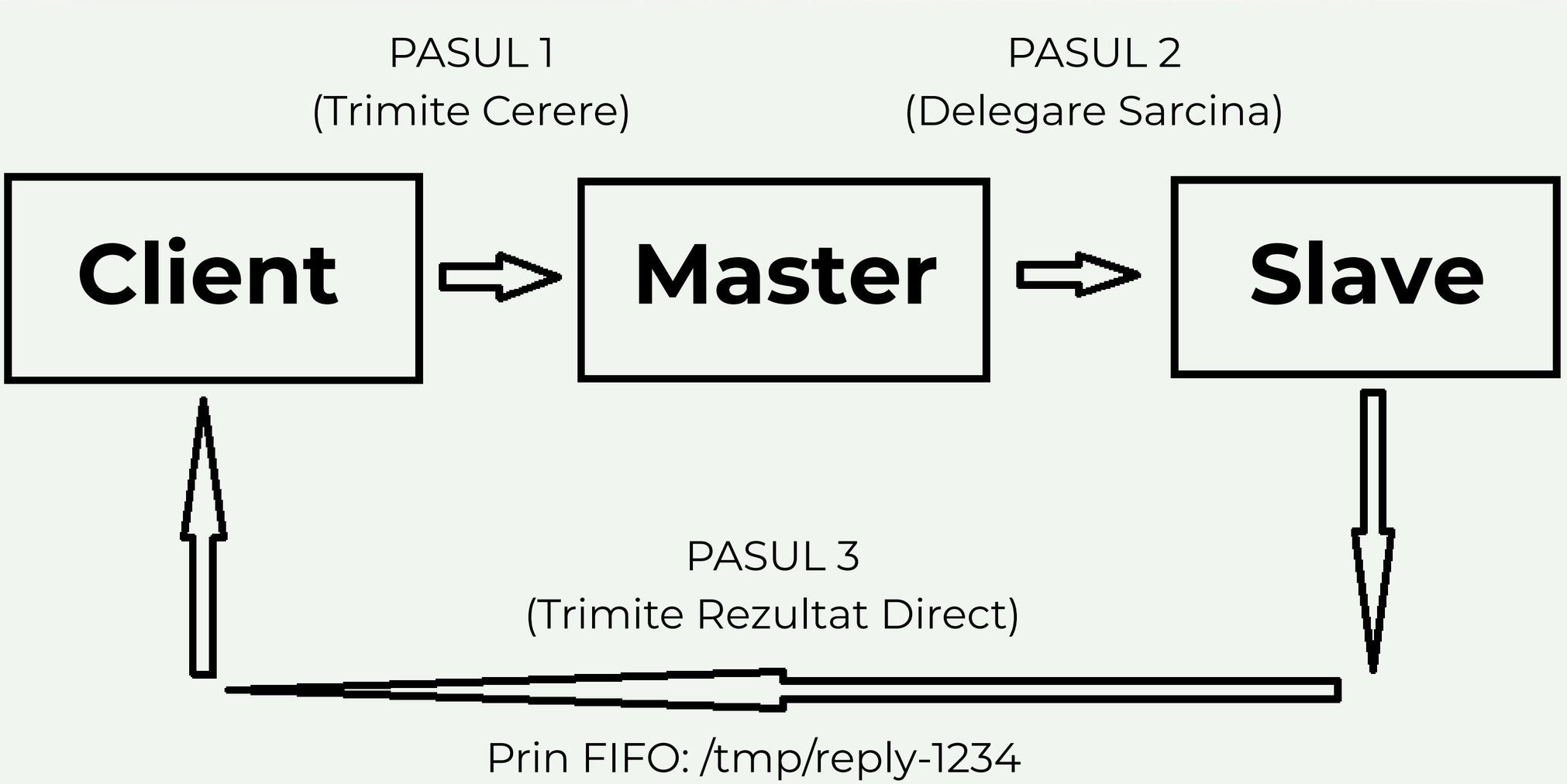
2. Master

- **Stă și ascultă** toate cererile venite de la clienți pe un canal principal.
- Nu dă toată munca unui singur om. **Împarte comenzile pe rând** (*circular*) la fiecare muncitor (*Slave*) disponibil. (*Round-Robin*)

3. Slave

- **Primesc comanda** de la Master și **o execută** imediat.
- **Trimit rezultatul** execuției (ceea ce ar fi apărut în mod normal pe ecran) direct în „cutia poștală” privată a clientului.

Designul sistemului



Designul sistemului

PASUL 1

Client → Master

Clientul trimite cererea în „Ghișeul Central” (*Master FIFO*). Mesajul include **PID-ul clientului** și **comanda dorită**.

PASUL 2

Master → Slave

Masterul acționează **ca un dispecer**. Acesta **citește cererea** și **o deleagă** către un Server Slave folosind algoritmul **Round-Robin** pentru a echilibra sarcina.

PASUL 3

Slave → Client

Aceasta este partea cea mai importantă a designului. Slave-ul **execută comanda și trimite rezultatul** direct către client, folosind FIFO-ul personal creat de acesta (/tmp/server-reply-XXXX).

Implementare

Tehnologii

- **Bash Scripting** (limbaj principal),
- **mkfifo** (pentru crearea canalelor de comunicație),
- **sed** (pentru extragerea PID-ului și a comenzii din mesaj),
- **eval** (permite execuția șirurilor de caractere primite ca fiind comenzi shell reale),
- **&** (utilizat de Master pentru a lansa serverele Slave în paralel)

Componente software:

Scripturile

- **master.sh,**
- **slave.sh,**
- **și client.sh.**

Implementare

Probleme tehnice întâmpinate

Gestionarea Citirii Blocate:

Problema: Scripturile rămân „înghețate” (block) la comanda read dacă nu există date în FIFO.

- *Soluția*: Utilizarea **buclelor while true** și asigurarea că **descriptorii de fișiere rămân deschiși** sau **se redeschid corect** pentru a menține serverele active

Izolarea Clienților:

Problema: Cum ne asigurăm că Rezultatul A nu ajunge la Clientul B?

- *Soluția*: **Generarea de FIFO-uri unice** folosind variabila de sistem $$ (PID-ul procesului), rezultând căi unice de tipul **/tmp/server-reply-XXXX**.

Protocolul de Mesaje:

Problema: Separarea comenzii de metadate.

- *Soluția*: Implementarea marcajelor **BEGIN-REQ și END-REQ** pentru a identifica clar unde începe și unde se termină o comandă în fluxul de date.

Experimente (Cum am testat?)

1. Configurația de test

Am utilizat o mașină virtuală cu sistem de operare Linux (Ubuntu) pentru a rula simultan Masterul și cei 3 Slaves.

2. Testarea concurenței

Am deschis mai multe terminale în paralel pentru a simula mai mulți clienți care trimit comenzi în același timp.

3. Verificarea Round-Robin

Am observat log-urile Masterului pentru a confirma că prima comandă a mers la Slave 1, a doua la Slave 2 și a treia la Slave 3.

4. Performanță și timp de răspuns

Sistemul a răspuns instantaneu pentru comenzi de bază (date, whoami) și a gestionat corect erorile (de exemplu, afișând mesajul de eroare în terminalul clientului dacă comanda era greșită).

Terminalul 1 (master.sh)

```
gabi@gabi-VirtualBox:~/Desktop/PROIECT$ chmod +x master.sh client.sh slave.sh
gabi@gabi-VirtualBox:~/Desktop/PROIECT$ ./master.sh
Waiting requests from /tmp/rshd_master_fifo
Slave 1 initialized (PID 6303)
Slave 2 initialized (PID 6304)
Slave 3 initialized (PID 6305)
Slave 1 initialized. Waiting commands from /tmp/rshd_slaves/slave_1
Slave 3 initialized. Waiting commands from /tmp/rshd_slaves/slave_3
Slave 2 initialized. Waiting commands from /tmp/rshd_slaves/slave_2
[master] Request from 6317: 'date'
[master] Sent to Slave 1 --> /tmp/rshd_slaves/slave_1
[slave] Slave 1 running command: 'date'
[slave] Slave 1: output sent to /tmp/server-reply-6317
[master] Request from 6334: 'whoami'
[master] Sent to Slave 2 --> /tmp/rshd_slaves/slave_2
[slave] Slave 2 running command: 'whoami'
[slave] Slave 2: output sent to /tmp/server-reply-6334
[master] Request from 6357: 'date; whoami'
[master] Sent to Slave 3 --> /tmp/rshd_slaves/slave_3
[slave] Slave 3 running command: 'date; whoami'
[slave] Slave 3: output sent to /tmp/server-reply-6357
[master] Request from 6376: 'cat fisier_care_nu_exista'
[master] Sent to Slave 1 --> /tmp/rshd_slaves/slave_1
[slave] Slave 1 running command: 'cat fisier_care_nu_exista'
[slave] Slave 1: output sent to /tmp/server-reply-6376
```

```
gabi@gabi-VirtualBox:~/Desktop/PROIECT$ ./client.sh date
Client 6317 --- FIFO: /tmp/server-reply-6317
Client 6317: Request sent. Waiting response...

--- RESULT OF [date] ---
Fri Jan  2 03:34:46 PM EET 2026
*------------------------------*
Client 6317: FIFO deleted.
gabi@gabi-VirtualBox:~/Desktop/PROIECT$
```

Terminalul 2 (client.sh)

Comanda **date**

Terminalul 3 (client.sh)

Comanda **whoami**

```
gabi@gabi-VirtualBox:~/Desktop/PROIECT$ ./client.sh whoami
Client 6334 --- FIFO: /tmp/server-reply-6334
Client 6334: Request sent. Waiting response...

--- RESULT OF [whoami] ---
gabi
*------------------------------*
Client 6334: FIFO deleted.
gabi@gabi-VirtualBox:~/Desktop/PROIECT$
```

```
gabi@gabi-VirtualBox:~/Desktop/PROIECT$ ./client.sh "date; whoami"
Client 6357 --- FIFO: /tmp/server-reply-6357
Client 6357: Request sent. Waiting response...

--- RESULT OF [date; whoami] ---
Fri Jan  2 03:35:23 PM EET 2026
gabi
*------------------------------*
Client 6357: FIFO deleted.
gabi@gabi-VirtualBox:~/Desktop/PROIECT$
```

Terminalul 4 (client.sh)

Comenzile **date** si **whoami**

Terminalul 5 (client.sh)

Comandă greșită

```
gabi@gabi-VirtualBox:~/Desktop/PROIECT$ ./client.sh "cat fisier_care_nu_exista"
Client 6376 --- FIFO: /tmp/server-reply-6376
Client 6376: Request sent. Waiting response...

--- RESULT OF [cat fisier_care_nu_exista] ---
cat: fisier_care_nu_exista: No such file or directory
*------------------------------*
Client 6376: FIFO deleted.
gabi@gabi-VirtualBox:~/Desktop/PROIECT$
```

Concluzii

Importanța IPC

Am înțeles practic cum procesele pot colabora folosind **named pipes** (FIFO) pentru a schimba date fără a fi rude între ele.

Gândirea distribuită

Am învățat cum să împărțim o sarcină mare (gestionarea multor clienți) în sarcini mai mici distribuite către mai mulți „muncitori” (Slaves).

Eficiența automatizării

Bash scripting-ul este un instrument extrem de puternic pentru a crea sisteme complexe de administrare cu foarte puține linii de cod.

Managementul resurselor

Am conștientizat importanța curățării sistemului (ștergerea FIFO-urilor la final) pentru a menține un mediu de lucru ordonat.